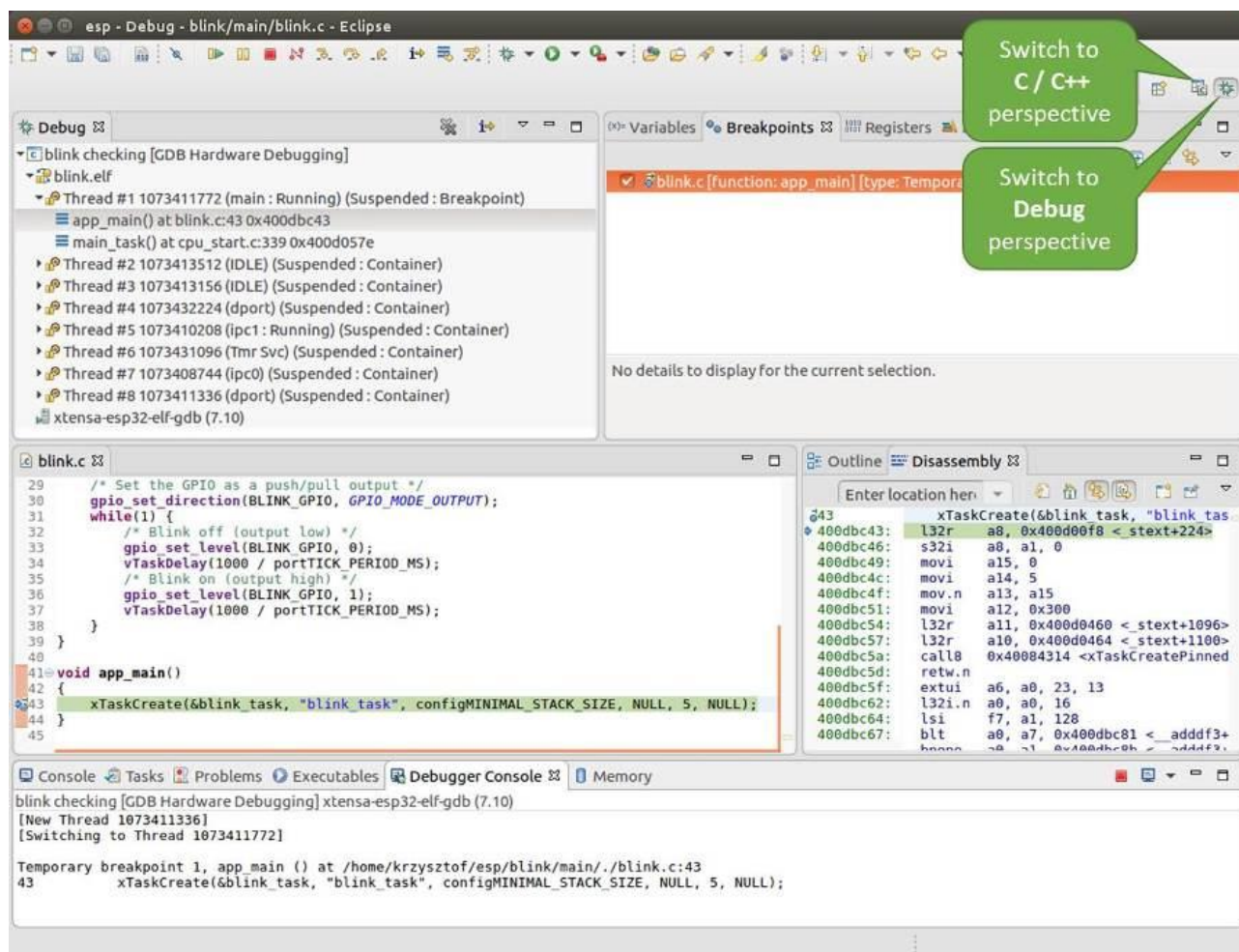


使用 Eclipse 的偵錯示例

請檢查目標板是否已經準備好，並載入了 [get-started/blink](#) 示例程式碼，然後按照 [使用 Eclipse 偵錯](#) 中介紹的步驟組態和啟動偵錯程式，最後選擇讓應用程式在 `app_main()` 建立的斷點處停止。



Eclipse 中的 Debug 檢視

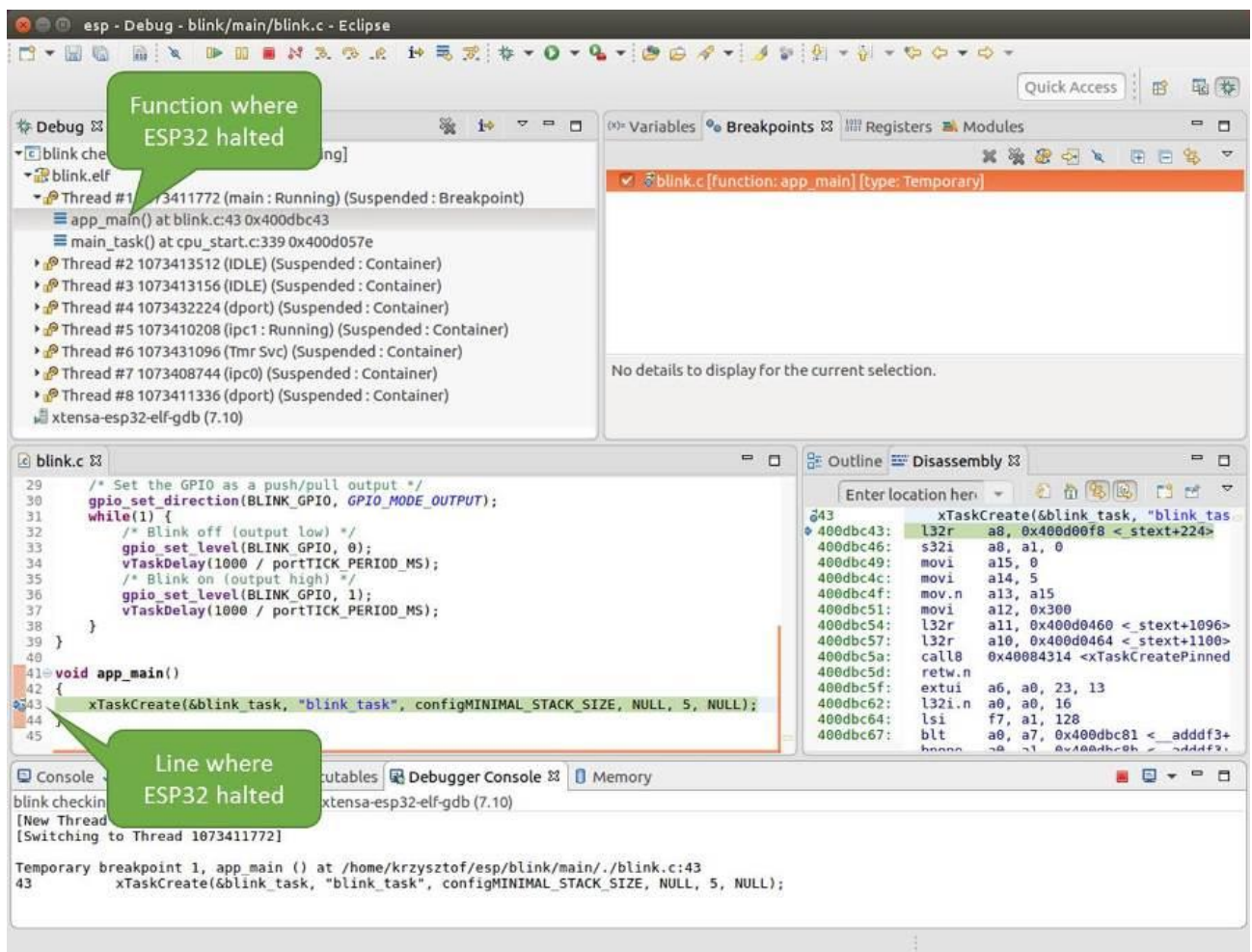
本小節的示例

1. [瀏覽程式碼，查看堆疊和執行緒](#)
2. [設定和清除斷點](#)
3. [手動暫停目標](#)
4. [單步執行程式碼](#)

5. [查看並設定記憶體](#)
6. [觀察和設定程序變數](#)
7. [設定條件斷點](#)

瀏覽程式碼，查看堆疊和執行緒

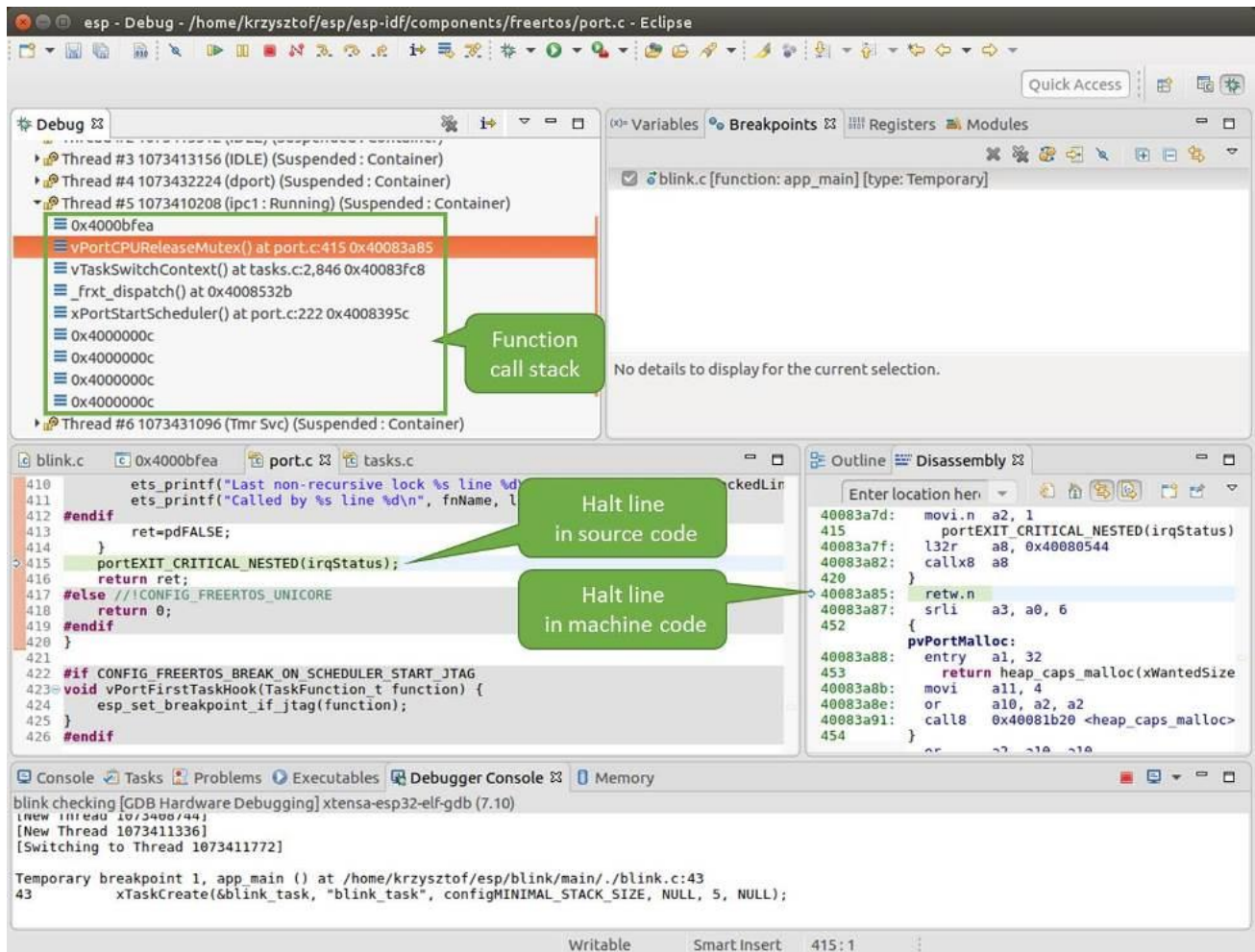
當目標暫停時，偵錯程式會在“Debug”窗口中顯示執行緒的列表，程序暫停的程式碼行在下面的另一個窗口中被高亮顯示，如下圖所示。此時板子上的 LED 停止了閃爍。



偵錯時目標停止

暫停的程序所線上程也會被展開，顯示函數呼叫的堆疊，它表示直到目標暫停所在程式碼行（下圖高亮處）為止的相關函數的呼叫關係。1 號執行緒下函數呼叫堆疊的第一行包含了最後一個呼叫的函數 `app_main()`，根據下一行顯示，它又是在函數 `main_task()` 中被呼叫的。堆疊的每一行還包含呼叫函數的檔案名稱和行號。通過點選每個堆疊的條目，在下面的窗口中，你將看到此檔案的內容。

通過展開執行緒，你可以瀏覽整個應用程式。展開 5 號執行緒，它包含了更長的函數呼叫堆疊，你可以看到函數呼叫旁邊的數字，比如 0x4000000c，它們代表未以原始碼形式提供的二進制程式碼所在的記憶體地址。



瀏覽函數呼叫堆疊

無論項目是以原始碼還是僅以二進制形式提供，在右邊一個窗口中，都可以看到反彙編後的機器程式碼。

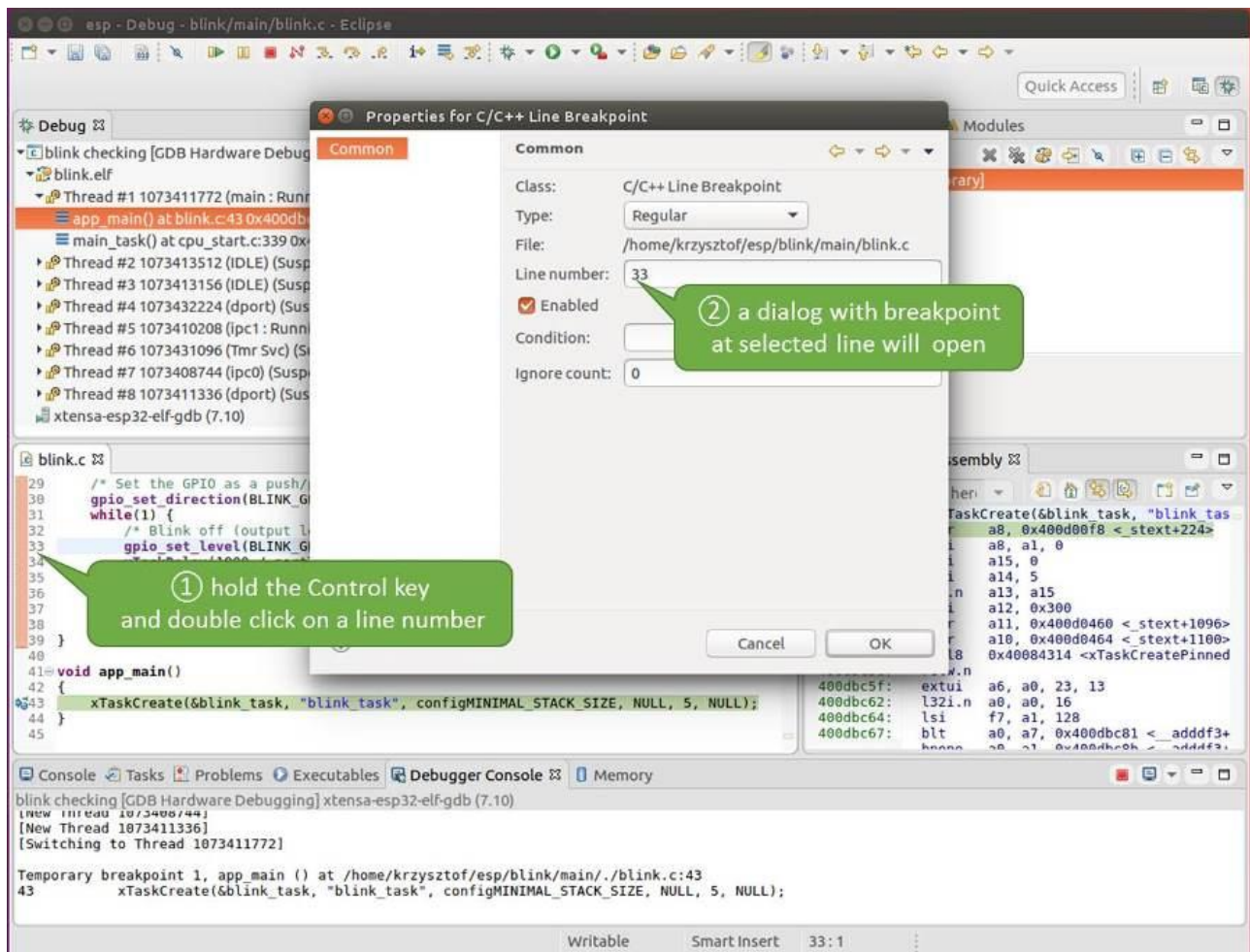
回到 1 號執行緒中的 `app_main()` 函數所在的 `blink.c` 原始碼檔案，下面的示例將會以該檔案為例介紹偵錯的常用功能。偵錯程式可以輕鬆瀏覽整個應用程式的程式碼，這給單步偵錯程式碼和設定斷點帶來了很大的便利，下面將一一展開討論。

設定和清除斷點

在偵錯時，我們希望能夠在關鍵的程式碼行停止應用程式，然後檢查特定的變數、記憶體、暫存器和外設的狀態。為此我們需要使用斷點，以便在特定某行程式碼處快速訪問和停止應

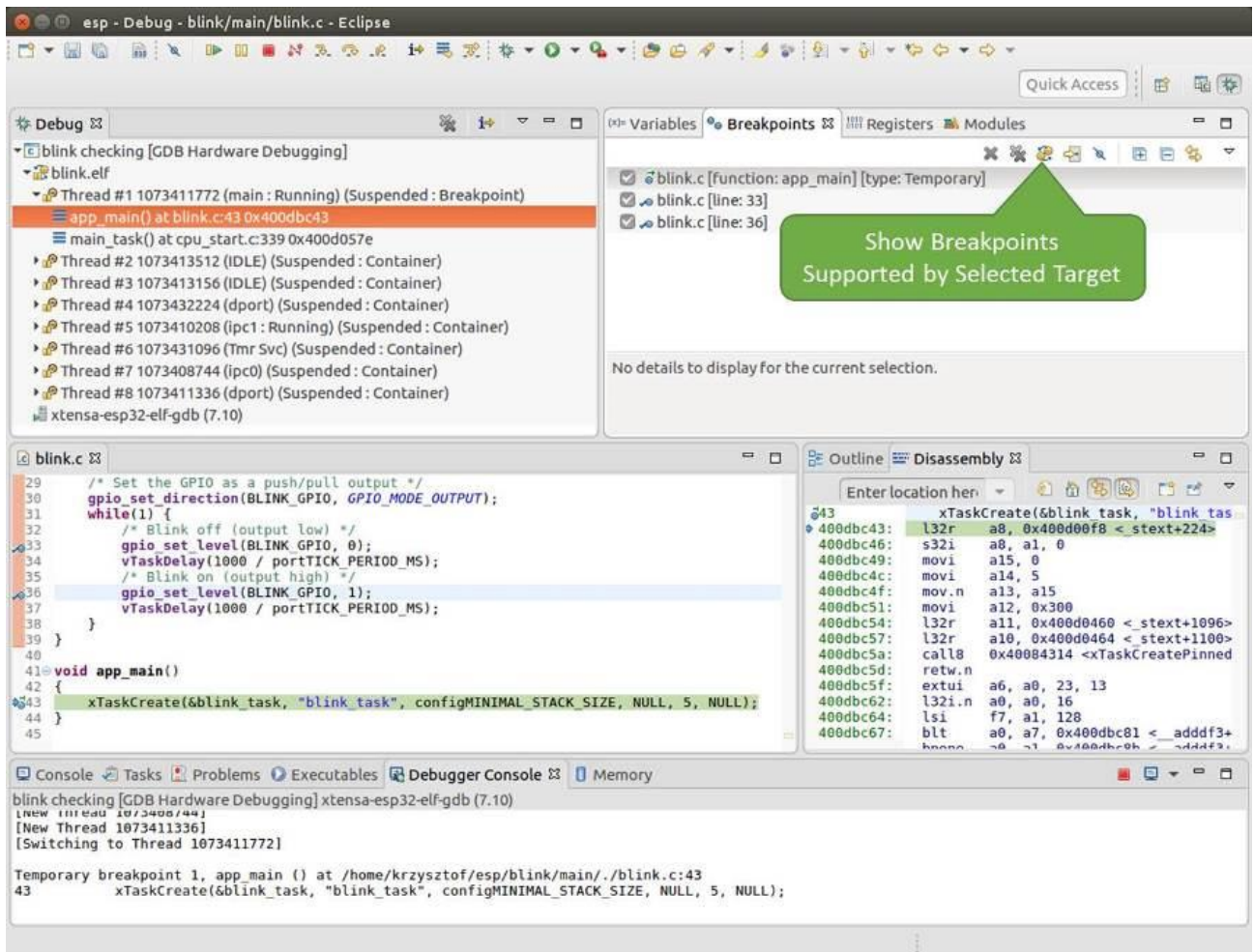
用程式。

我們在控制 LED 狀態發生變化的兩處程式碼行分別設定一個斷點。基於以上程式碼列表，這兩處分別為第 33 和 36 程式碼行。按住鍵盤上的“Ctrl”鍵，連按兩下 blink.c 檔案中的行號 33，並在彈出的對話方塊中點選“OK”按鈕進行確定。如果你不想看到此對話方塊，連按兩下行號即可。執行同樣操作，在第 36 行設定另外一個斷點。



設定斷點

斷點的數量和位置資訊會顯示在右上角的“斷點”窗口中。點選“Show Breakpoints Supported by Selected Target”圖示可以刷新此列表。除了剛才設定的兩個斷點外，列表中可能還包含在偵錯程式啟動時設定在 app_main() 函數處的臨時斷點。由於最多隻允許設定兩個斷點（詳細資訊請參閱[可用的斷點和觀察點](#)），你需要將其刪除，否則偵錯會失敗。



設定了三個斷點 / 最多允許兩個斷點

點選“Resume”（如果“Resume”按鈕是灰色的，請先點選 8 號執行緒的 blink_task() 函數）後處理器將開始繼續運行，並在斷點處停止。再一次點選“Resume”按鈕，使程序再次運行，然後停在第二個斷點處，依次類推。

每次點選“Resume”按鈕恢復程式執行後，都會看到 LED 切換狀態。

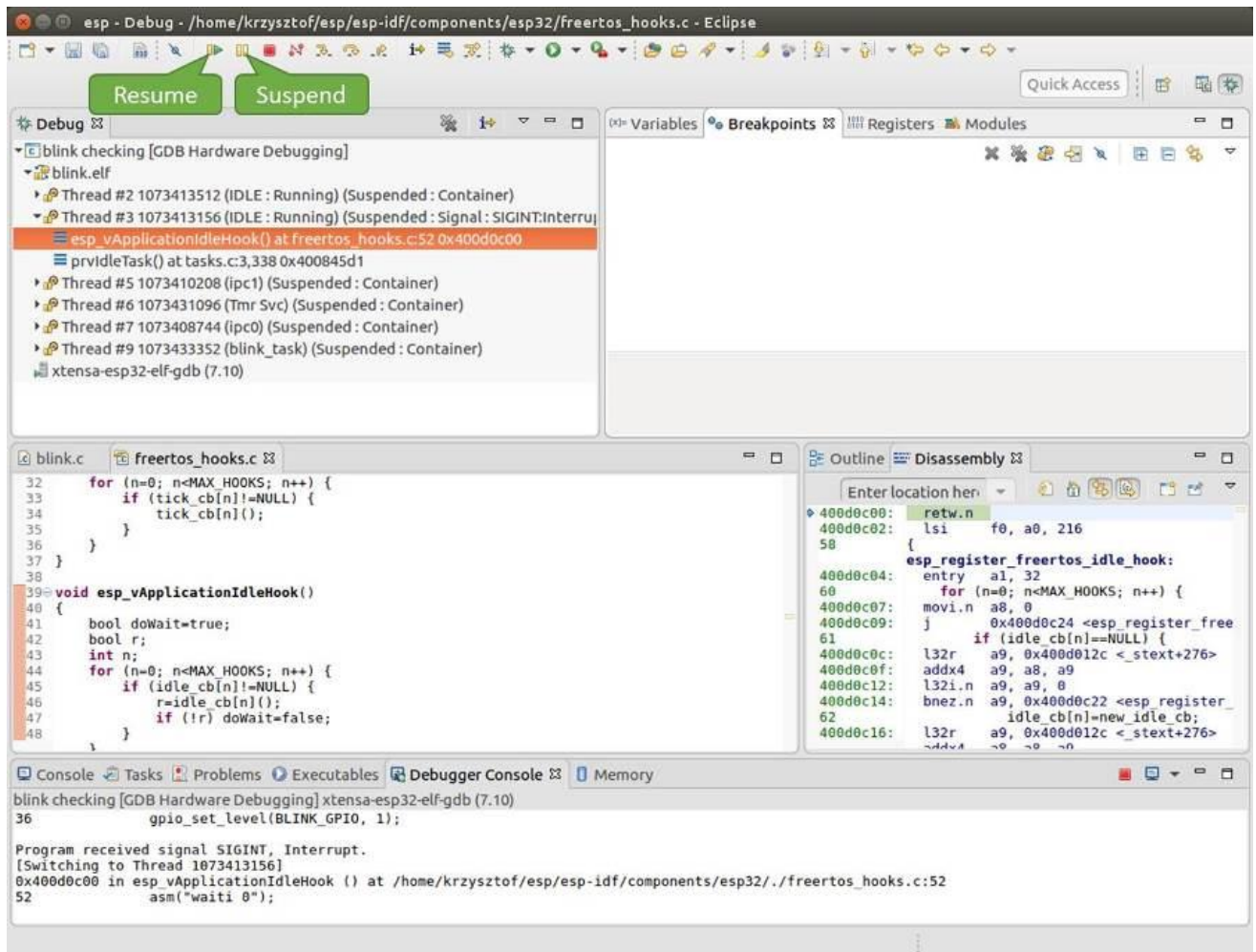
更多關於斷點的資訊，請參閱[可用的斷點和觀察點](#)和[關於斷點的補充知識](#)。

手動暫停目標

在偵錯時，你可以恢復程式執行並輸入程式碼等待某個事件發生或者保持無限循環而不設定任何斷點。後者，如果想要返回呼試模式，可以通過點選“Suspend”按鈕來手動中斷程序的運行。

在此之前，請刪除所有的斷點，然後點選“Resume”按鈕。接著點選“Suspend”按鈕，應用程式會停止在某個隨機的位置，此時 LED 也將停止閃爍。偵錯程式將展開執行緒並高亮顯

示停止的程式碼行。



手動暫停目標

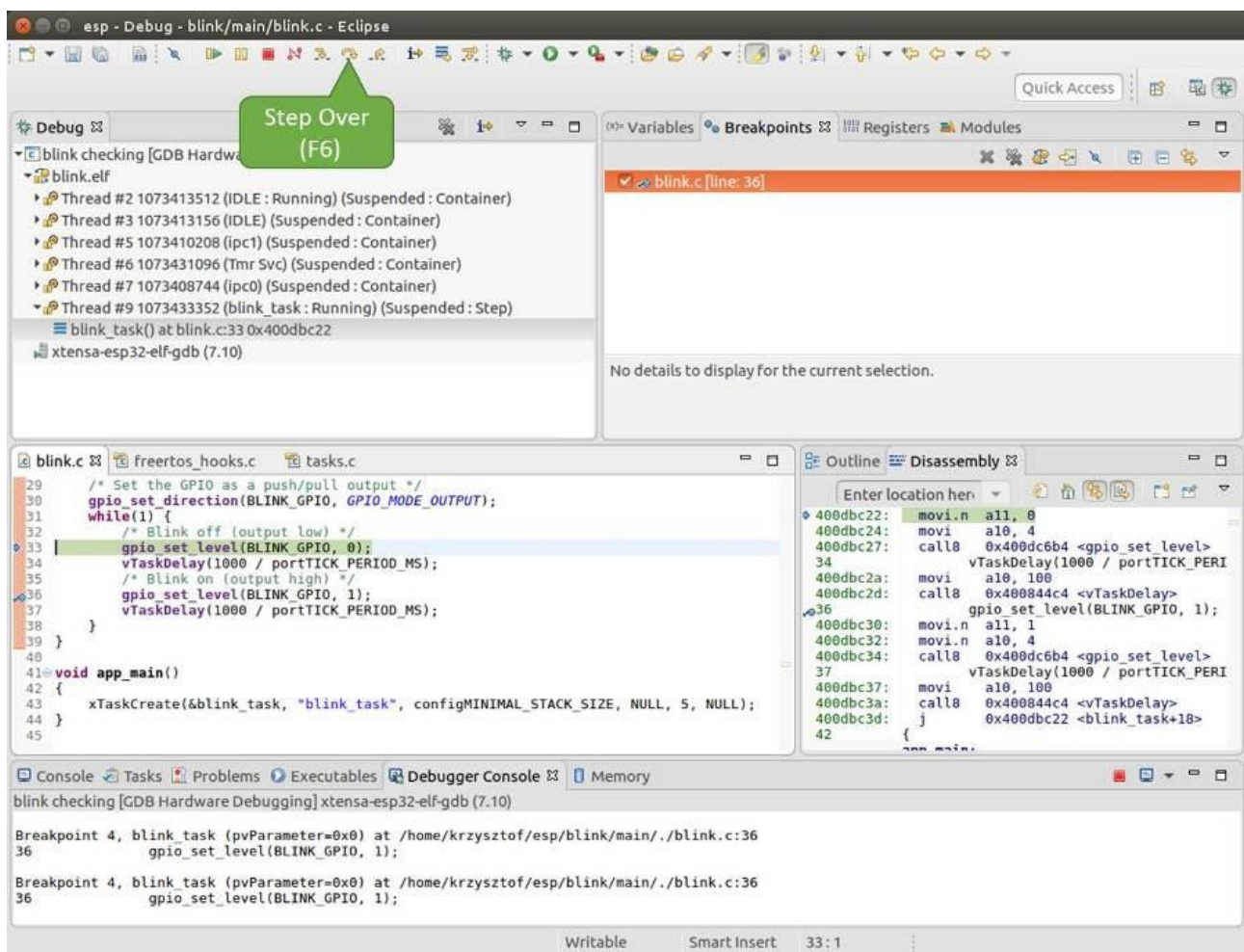
在上圖所示的情況中，應用程式已經在 `freertos_hooks.c` 檔案的第 52 行暫停運行，現在你可以通過點選“Resume”按鈕再次將其恢復運行或者進行下面要介紹的偵錯工作。

單步執行程式碼

我們還可以使用“Step Into (F5)”和“Step Over (F6)”命令單步執行程式碼，這兩者之間的區別是執行“Step Into (F5)”命令會進入呼叫的子程序，而執行“Step Over (F6)”命令則會直接將子程序看成單個原始碼行，單步就能將其運行結束。

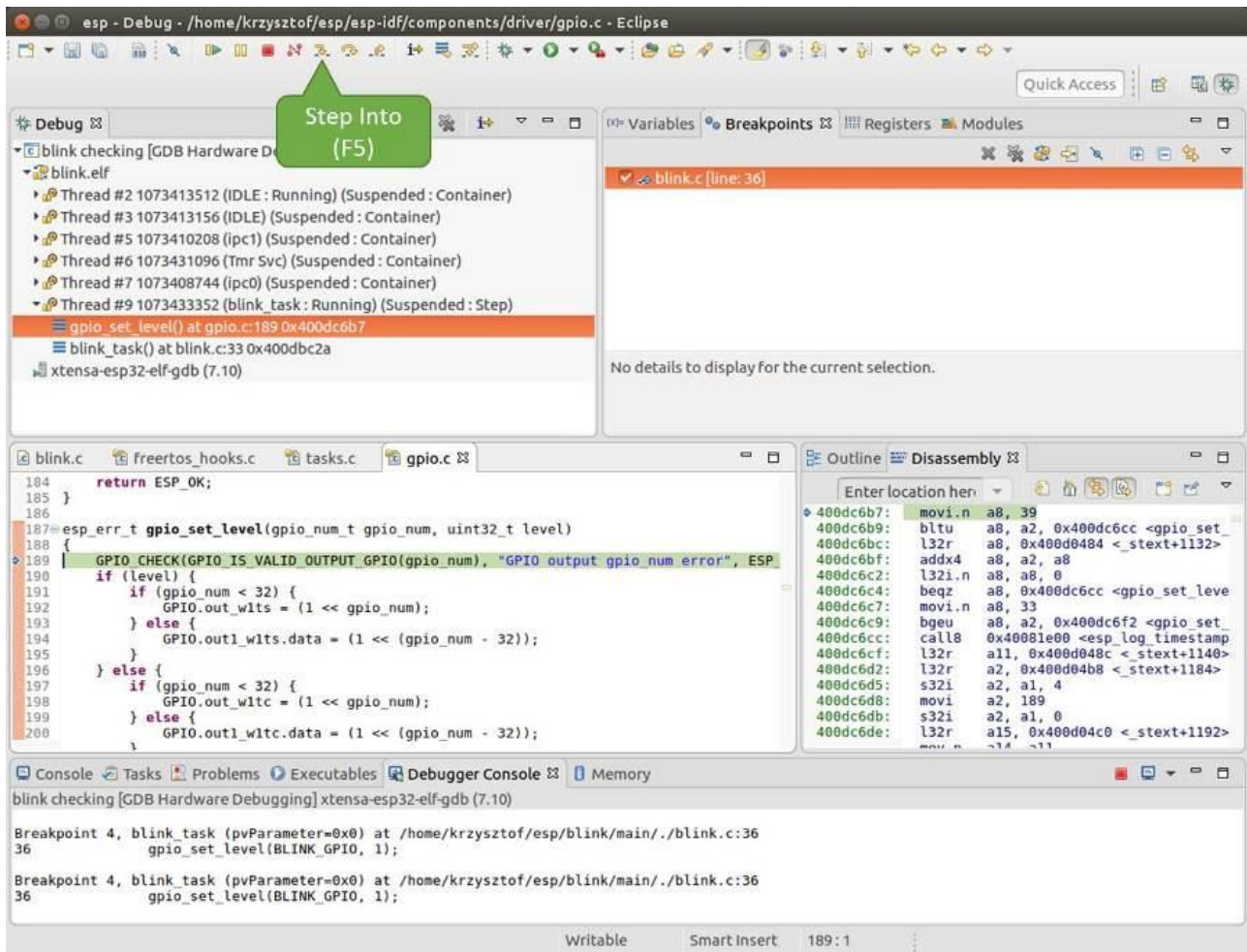
在繼續演示此功能之前，請參照上文所述確保目前只在 `blink.c` 檔案的第 36 行設定了一個斷點。

按下 F8 鍵讓程序繼續運行然後在斷點處停止運行，多次按下“Step Over (F6)”按鈕，觀察偵錯程式是如何單步執行一行程式碼的。



使用“Step Over (F6)”單步執行程式碼

如果你改用 "Step Into (F5)"，那麼偵錯程式將會進入呼叫的子程序內部。



使用“Step Into (F5)”單步執行程式碼

在上述例子中，偵錯程式進入 `gpio_set_level(BLINK_GPIO, 0)` 程式碼內部，同時程式碼窗口快速切換到 `gpio.c` 驅動檔案。

請參閱 [“next”命令無法跳過子程序的原因](#) 文件以瞭解 `next` 命令的潛在侷限。

查看並設定記憶體

要顯示或者設定記憶體的內容，請使用“偵錯”檢視中位於底部的“Memory”選項卡。

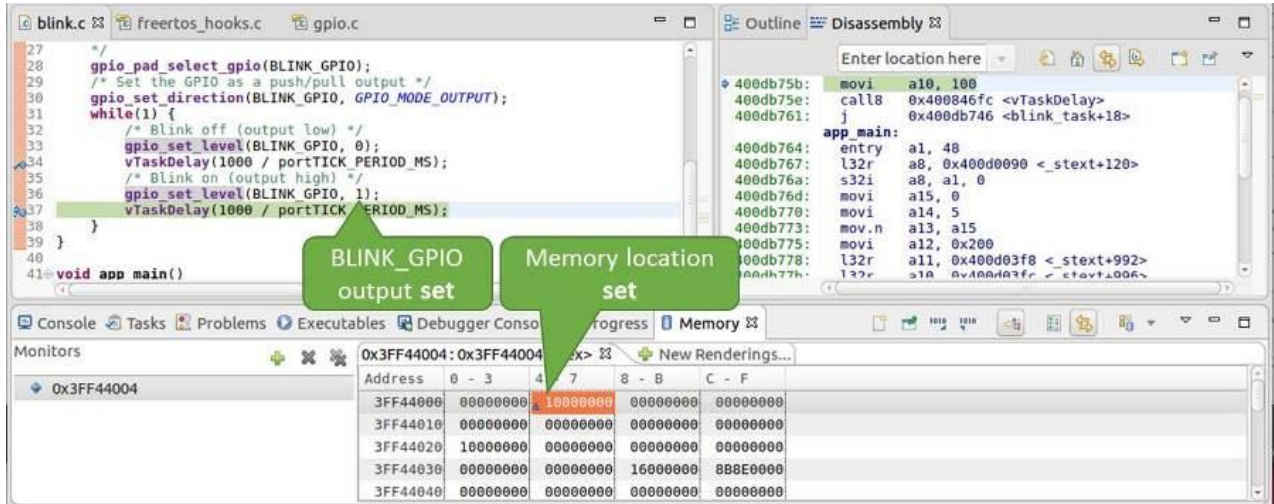
在“Memory”選項卡下，我們將在記憶體地址 `0x3FF44004` 處讀取和寫入內容。該地址也是 `GPIO_OUT_REG` 暫存器的地址，可以用來控制（設定或者清除）某個 `GPIO` 的電平。

關於該暫存器的更多詳細資訊，請參閱 *ESP32 技術參考手冊* > *IO MUX* 和 *GPIO Matrix (GPIO, IO_MUX)* [PDF] 章節。

同樣在 `blink.c` 項目檔案中，在兩個 `gpio_set_level` 語句的後面各設定一個斷點，點選

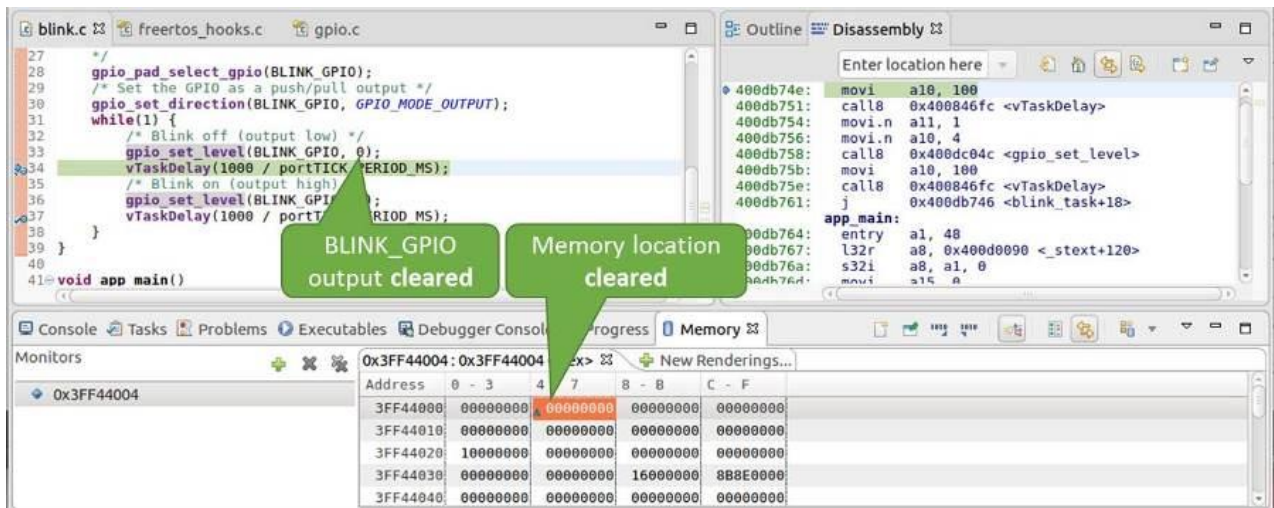
“Memory” 選項卡，然後點選 “Add Memory Monitor” 按鈕，在彈出的對話方塊中輸入 0x3FF44004。

按下 F8 按鍵恢復程式執行，並觀察 “Monitor” 選項卡。



觀察記憶體地址 0x3FF44004 處的某個位元被置高

每按一下 F8，你就會看到在記憶體 0x3FF44004 地址處的一個位元被翻轉（並且 LED 會改變狀態）。



觀察記憶體地址 0x3FF44004 處的某個位元被置低

要修改記憶體的數值，請在 “Monitor” 選項卡中找到待修改的記憶體地址，如前面觀察的結果一樣，輸入特定位元翻轉後的值。當按下 Enter 鍵後，將立即看到 LED 的狀態發生了改變。

觀察和設定程序變數

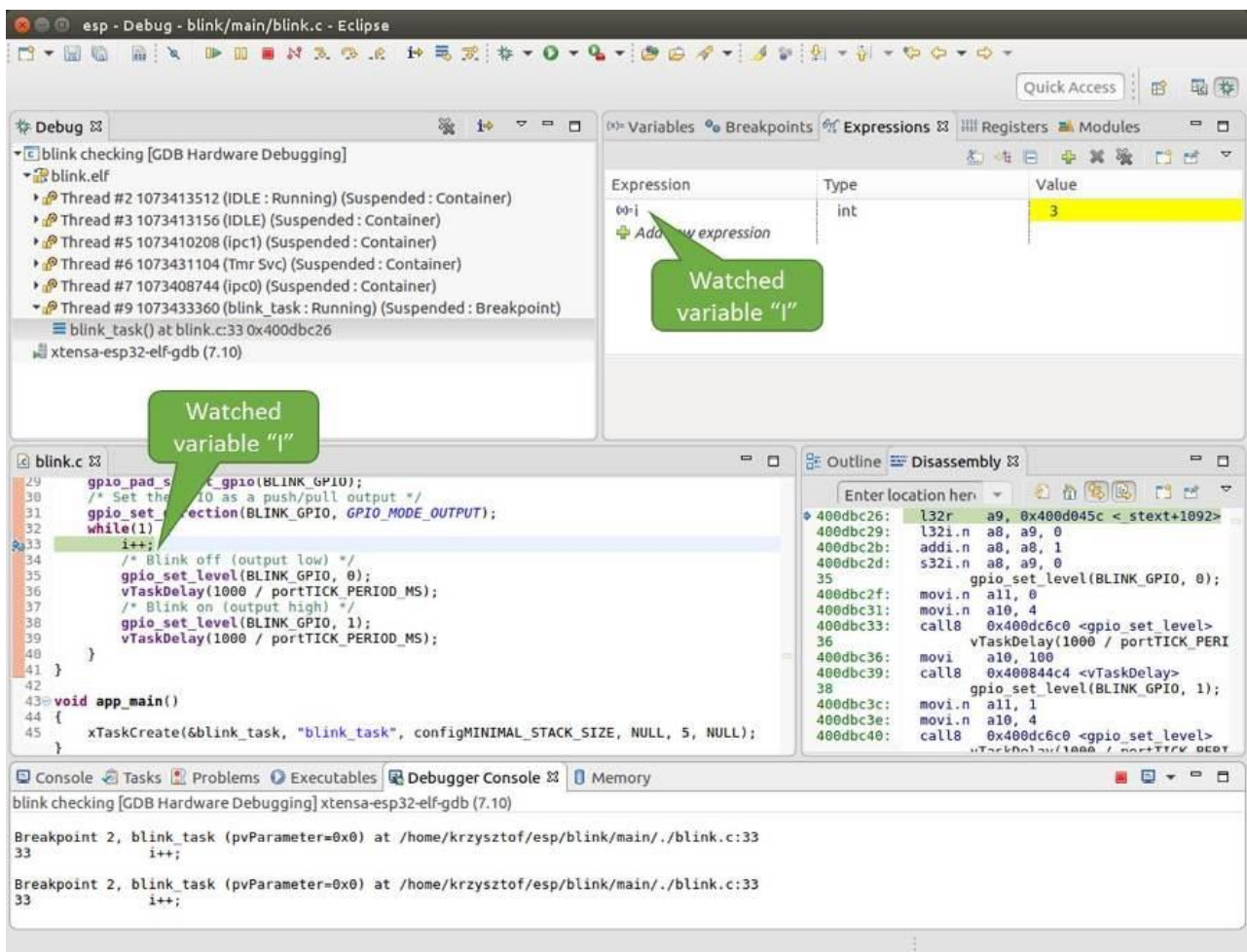
常見的偵錯任務是在程式執行期間檢查程序中某個變數的值，為了演示這個功能，更新 `blink.c` 檔案，在 `blink_task` 函數的上面新增一個全域變數的聲明 `int i`，然後在 `while(1)` 裡新增 `i++`，這樣每次 LED 改變狀態的時候，變數 `i` 都會增加 1。

退出偵錯程式，這樣就不會與新程式碼混淆，然後重新建構並燒寫程式碼到 ESP32 中，接著重啟偵錯程式。注意，這裡不需要我們重啟 OpenOCD。

一旦程序停止運行，在程式碼 `i++` 處新增一個斷點。

下一步，在“Breakpoints”所在的窗口中，選擇“Expressions”選項卡。如果該選項卡不存在，請在頂部菜單欄的 `Window > Show View > Expressions` 中新增這一選項卡。然後在該選項卡中點選“Add new expression”，並輸入 `i`。

按下 F8 繼續運程序，每次程序停止時，都會看到變數 `i` 的值在遞增。

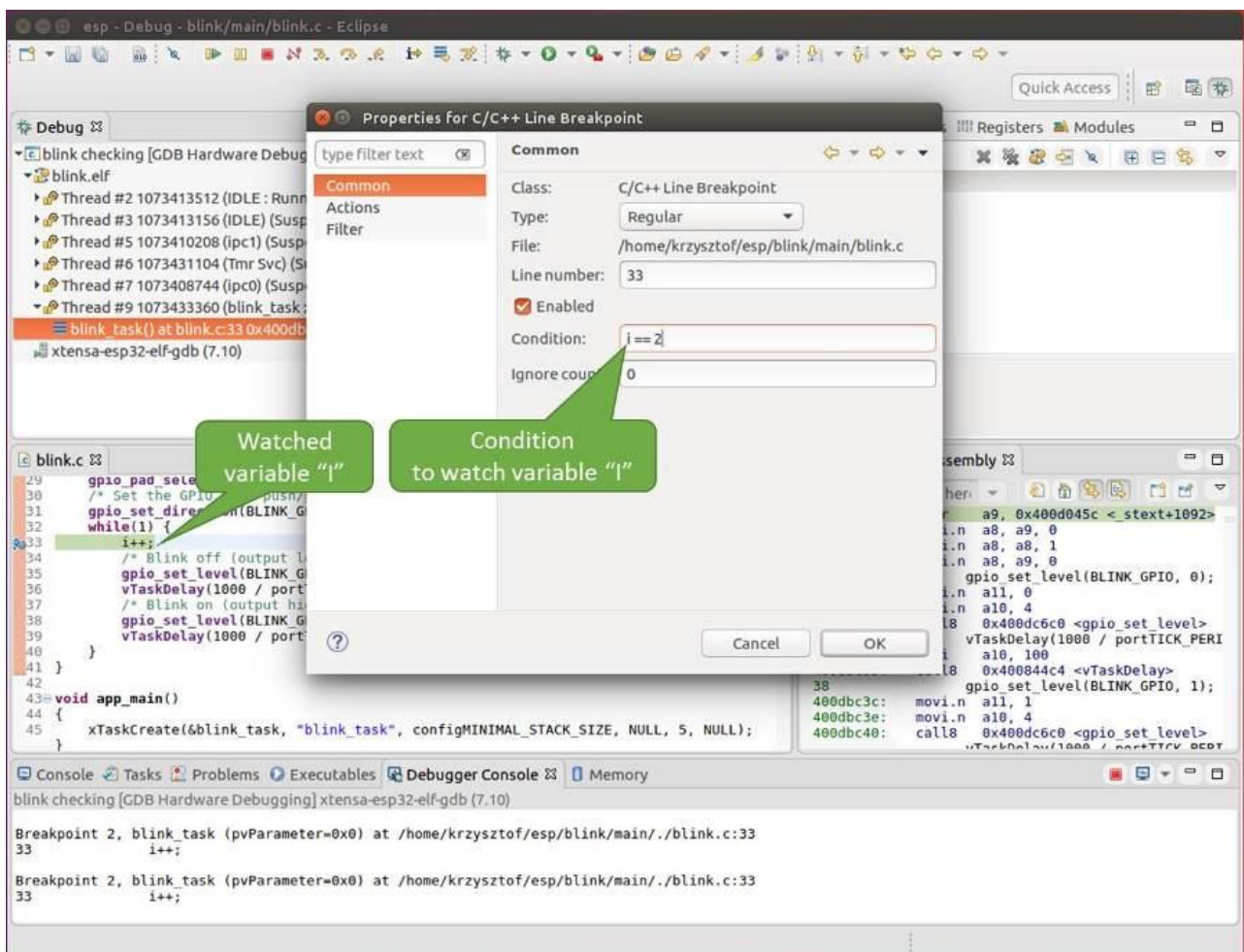


觀察程序變數 “i”

如想更改 `i` 的值，可以在 “Value” 一欄中輸入新的數值。按下 “Resume (F8)” 後，程序將從新輸入的數字開始遞增 `i`。

設定條件斷點

接下來的內容更為有趣，你可能想在一定條件滿足的情況下設定斷點，然後讓程序停止運行。右擊斷點打開上下文菜單，選擇 “Breakpoint Properties”，將 “Type:” 改選為 “Hardware” 然後在 “Condition:” 一欄中輸入條件表示式，例如 `i == 2`。



設定條件斷點

如果當前 `i` 的值小於 2（如果有需要也可以更改這個閾值）並且程序被恢復運行，那麼 LED 就會循環閃爍，直到 `i == 2` 條件成立，最後程序停止在該處。